

Implement Client Script & UI Policy (Incident)

Based on the Incident form configuration, testing and observations of the ServiceNow project

SCENARIO Configuring, enforcing and testing High-impact Incident controls in ServiceNow	Entice	Enter	Engage	Exit	Extend
	How does someone initially become aware of this process?	What do people experience as they begin the process?	In the core moments in the process, what happens?	What do people typically experience as the process finishes?	What happens after the experience is over?
Steps What does the person (or group) typically experience?	Identify gap in Incident data quality Log in with admin / configuration access Open System UI → UI Policies	Click New to create a UI Policy Name it High Impact Control; table Incident; condition Impact is 1 - High Add actions: Assignment group mandatory, Urgency read-only; Reverse if false = true	Create onChange script on Impact to auto-set Urgency Create onSubmit script to block save when Assigned To is empty Create onCellEdit script to block State edits in list	Create a new Incident with Impact = High Leave Assigned To empty and click Submit Fill Assigned To and submit successfully	Change Impact from High to Medium and verify reverse Double-click State in the Incident list Update State from the Incident form
Interactions What interactions do they have at each step along the way? People · Places · Things	Admin ↔ application navigator (filter 'UI pol') UI Policies list view, 2,842 records	UI Policy form – When to Apply tab UI Policy Actions related list	System UI → Client Scripts form Script editor with Isolate script enabled	Incident → Create New form Submit button; error box shown on Assigned To field	Incident list (Incident → Open / All) Incident form Update button; alert pop-up
Goals & motivations At each step, what is a person's primary goal or motivation?	Help me ensure Incidents are triaged with complete data Help me find the right configuration area	Help me apply extra controls only when Impact is High Help me revert changes when the condition is no longer true	Help me automate Urgency so it is never missed Help me stop invalid saves and list edits	Help me confirm bad data is actually blocked Help me confirm valid records still save	Help me confirm the reverse condition works Help me keep State controlled in lists but editable on the form
Positive moments What steps does a typical person find enjoyable, productive or delightful?	Clear problem statement and objective Simple, familiar navigation	UI Policies are declarative – no code needed Reverse if false undoes changes automatically	Urgency auto-sets with an info message Scripts are short and lightweight	Error box clearly points to Assigned To Valid Incident saves without errors	Urgency and Assigned To behave as expected after reverse Form edit of State works while list edit is blocked
Negative moments What steps does a typical person find frustrating, confusing or time-consuming?	Manual checks lead to incomplete or inconsistent data Hard to search among many existing UI Policies	Easy to pick wrong field, operator or value Assignment group (policy) vs Assigned To (script) can confuse	Wrong script type or field name stops script firing Client-side rules can be bypassed by imports / APIs	User blocked on save may not know why if message is unclear	Urgency stays High after Impact is lowered to Medium Alert pop-up interrupts list editing
Areas of opportunity How might we make each step better? What ideas do we have?	Document business rules before configuring Capture changes in an Update Set	Extend policy to more fields (e.g. Work notes) Use UI Policy conditions with more criteria	Add server-side Business Rule / Data Policy as a backstop Replace alert() with g_form messages	Automate these test cases with ATF Show a summary of why the save was blocked	Reset Urgency when Impact is lowered Allow role-based exceptions for list editing